

Building a Rust-Like Survival Game From Scratch — Complete Engineering Plan

1. Purpose and Scope

This document defines a complete, implementation-oriented engineering plan for a small, senior-heavy team building a persistent, multiplayer, Rust-like survival game from the ground up with AI-assisted development.

The goal is not merely to list gameplay systems. The plan establishes the architectural contracts, server-authority rules, data ownership, performance boundaries, persistence strategy, testing strategy, operational requirements, live-ops model, and AI-assisted development workflow needed to reach — and sustain — a production-ready game without accumulating avoidable technical debt.

The plan is organized around the systems that make this genre work, grouped into four tiers:

- **Core simulation systems:** networking, building/stability, world generation, survival/environment, crafting/progression, combat, vehicles, audio.
- **World-continuity systems:** persistence, wipes/seasons, performance/scaling, AI/NPCs, modding.
- **Player-facing and trust systems:** interaction, onboarding/localization/accessibility, ownership/permissions, anti-cheat, admin/moderation, security, legal/compliance.
- **Delivery and operations systems:** concurrency/distributed servers, server lifecycle, observability, analytics, testing, CI/CD, client distribution.

The central engineering principle running through all of it: **AI accelerates implementation, testing, documentation, tooling, and iteration, while senior engineers retain ownership of architecture and correctness-critical decisions.** This is treated as a governance rule, not a suggestion — see Section 30 and Section 37.

Production assumptions

Unless a product decision overrides them, this document assumes:

- Dedicated authoritative game servers
- Persistent worlds, operating on a recurring wipe/season cycle
- Multiple concurrent players per world/server
- Client-side prediction for responsive movement
- Server-side validation for gameplay-affecting actions
- Chunked world representation
- Data-driven content
- Versioned persistence
- Server-side anti-cheat validation
- Automated tests and CI
- Horizontal server deployment as population grows
- A post-launch operating model (patch cadence, live events, community moderation), not just a launch date

- AI-assisted implementation with mandatory human review

Where exact values are presented below, they are **starting points, not final game-balance or capacity decisions**.

2. Foundational Decisions

These decisions constrain almost every downstream system and should be approved before substantial implementation begins. **The engine decision (row 1) should be treated as the single highest-priority open decision** — nearly every other row depends on it, and it should be forced to a firm answer before Phase 0 begins, not left open indefinitely.

Decision	Recommended baseline	Why
Engine	Unity with modular MonoBehaviours and plain C# domain logic; introduce ECS only if profiling demonstrates a need	Fits the existing Unity project and offline-first milestone while keeping gameplay rules testable and migration options open. Commit to one before Phase 0.
Networking	Dedicated authoritative server + client prediction + reconciliation	Prevents client-authority exploits and avoids a later networking rewrite
Server tick	Fixed simulation tick, initially 30 or 60 Hz depending on combat requirements	Makes movement, combat, rewind, and replication deterministic and testable
World	Seeded chunk/tile world	Enables streaming, persistence deltas, and spatial queries
Entity model	ECS or strongly componentized entity registry	Avoids tightly coupled gameplay objects
Persistence	Versioned Protobuf/FlatBuffers/MessagePack payloads	Supports schema evolution and compact storage
Database	Relational metadata DB + object/blob storage for large snapshots where appropriate	Separates transactional player data from large world data
Cache	Redis or equivalent only where measured need exists	Useful for sessions, distributed locks, queues, and hot metadata; avoid making it the source of truth
Authentication	External/platform identity converted to a server-issued player identity	Keeps gameplay identity independent from device-specific identifiers

Decision	Recommended baseline	Why
Deployment	Containerized dedicated servers	Consistent environments and repeatable deployment
Observability	Structured logs + metrics + traces + gameplay telemetry	Required to diagnose live-server failures
Content	Data-driven item, recipe, weapon, biome, and monument definitions	Enables designers and AI tooling to iterate without code changes
Randomness	Central seeded RNG service per deterministic system	Prevents accidental nondeterminism
Source control	Git + protected main branch + CI	Required for a small team moving quickly with AI-generated code
API contracts	Explicit versioned contracts	Prevents client/server and persistence compatibility problems
Modding	Decide in/out before content systems are built (Section 17)	Retrofitting mod hooks is far more expensive than designing them in

2A. Unity Phase-by-Phase Development Checklist

This section turns the above engineering plan into a practical Unity production roadmap. It is designed to keep the project stable in an offline-first build before introducing multiplayer and dedicated authoritative systems.

Phase 0 — Foundations and project setup

Objective

Build a clean Unity runtime foundation and prove that the game can boot, save, and recover safely.

Checklist

- ☐ Create Unity project structure: Core, Gameplay, World, Save, Systems, UI, Data, Testing.
- ☐ Configure input, fixed timestep, serialization, and build settings.
- ☒ Decide engine architecture: modular MonoBehaviours with plain C# domain logic; revisit ECS only if profiling justifies it.
- ☐ Create boot scene and startup bootstrap flow.
- ☐ Implement structured logger and debug tools.
- ☒ Add versioned save/load system with backup handling.
- ☐ Define player data, world data, and metadata boundaries.
- ☒ Implement deterministic RNG service.
- ☐ Add simple telemetry and event logging.
- ☐ Build CI basics for Unity build/test automation.

- ☒ Create a local test scene to validate runtime systems.

Exit criteria

- ☒ The game boots cleanly in editor.
 - ☒ Save and reload works repeatedly.
 - ☒ Data versioning is enforced.
 - ☐ Logs and telemetry are visible and structured.
 - ☐ The project is stable enough to begin gameplay work.
-

Phase 1 — Core survival loop

Objective

Deliver the basic survival gameplay loop that the rest of the project depends on.

Checklist

- ☐ Implement player controller: move, sprint, jump, crouch, collision checks.
- ☐ Build resource gathering system with resource node definitions and interaction flow.
- ☐ Implement inventory model: item IDs, stacks, equipment, containers.
- ☐ Implement crafting system with recipe validation and ingredient consumption.
- ☐ Create hunger, thirst, temperature, and health systems.
- ☐ Build day/night cycle and world-time manager.
- ☐ Add basic world interaction: pickup, use, place, destroy.
- ☐ Implement death and respawn flow.
- ☐ Save after key progression events.
- ☐ Add debug UI for inventory and survive stats.

Exit criteria

- ☐ A player can gather, store, craft, and use items.
 - ☐ Hunger/thirst/temperature systems are stable.
 - ☐ Save/load preserves essential player state.
 - ☐ The core loop is playable offline before network work begins.
-

Phase 2 — Building and structure simulation

Objective

Deliver structural placement, validity checks, and persistence for bases and building pieces.

Checklist

- ☐ Define structure schema: health, placement, owner, support state, and metadata.
- ☐ Build placement validation pipeline.
- ☐ Implement build preview visuals and rules.

- ☐ Add placement permissions and resource validation.
- ☐ Add structure persistence and reload behavior.
- ☐ Implement structure destruction and repair flows.
- ☐ Build support/stability system with propagation logic.
- ☐ Add large-base stress tests and invalid-placement tests.
- ☐ Confirm structure state survives save/load and world restarts.

Exit criteria

- ☐ Placement is valid and deterministic.
- ☐ Structural collapse logic is stable.
- ☐ Base construction is playable without corruption.
- ☐ Structure state persists cleanly.

Phase 3 — Combat and damage loop

Objective

Create a stable and testable combat system aligned with server-authoritative rules.

Checklist

- ☐ Define weapon data: fire rate, range, damage, reload, ammo, spread.
- ☐ Implement input and attack flow.
- ☐ Decide on hitscan or projectile model and commit to it.
- ☐ Build damage calculation with hit validation and fall damage if applicable.
- ☐ Implement death flow and respawn timers.
- ☐ Add hit effects, impact feedback, and recoil.
- ☐ Add combat telemetry and event logging.
- ☐ Validate against lag and delayed input scenarios.
- ☐ Test attack consistency from multiple angles and distances.

Exit criteria

- ☐ Combat is deterministic and reproducible.
- ☐ Damage values are validated before state changes apply.
- ☐ Death/respawn flow is consistent.
- ☐ The system is ready for server-authoritative upgrade.

Phase 4 — Procedural world generation

Objective

Generate a world that is deterministic, stable, and compatible with persistence and streaming.

Checklist

- ☐ Create world generation seed and version system.
- ☐ Implement terrain, biome, and weather rules.
- ☐ Add resource distribution logic.
- ☐ Place monuments, spawns, and safe zones.
- ☐ Generate road or traversal paths if required.
- ☐ Build world chunk lifecycle: load, unload, persist.
- ☐ Add validation pass for invalid spawn and resource placement.
- ☐ Track world generation metadata for later migration.

Exit criteria

- ☐ The same seed and version yields the same world.
- ☐ Terrain and resources are valid and traversable.
- ☐ Streaming and chunk boundaries remain stable.

Phase 5 — Progression, content, and deeper survival systems

Objective

Add the progression and content systems that make the game feel like a complete survival sandbox.

Checklist

- ☐ Build crafting stations and unlock system.
- ☐ Add item progression and recipe gating.
- ☐ Expand content via data-driven tables.
- ☐ Add hostile creatures or NPC behavior if in scope.
- ☐ Build tutorial and onboarding flow.
- ☐ Add progression persistence and state tracking.
- ☐ Validate pacing of resource acquisition and upgrades.

Exit criteria

- ☐ The game progression loop is comprehensible and consistent.
- ☐ Unlocks and recipes work across sessions.
- ☐ New content can be added without changing core gameplay code.

Phase 6 — Scale, optimization, and system hardening

Objective

Ensure the project can support many active entities and long runs without instability.

Checklist

- ☐ Profile memory, GC, active entities, and chunk streaming.
- ☐ Add object pooling for repeated effects and entities.

- ☐ Measure simulation cost per tick.
- ☐ Stress test building, combat, and world streaming separately.
- ☐ Build performance dashboards and telemetry alerts.
- ☐ Validate save times and crash recovery behavior.
- ☐ Audit anti-cheat and validation assumptions.

Exit criteria

- ☐ The project has a documented performance budget.
- ☐ No critical instability appears under load.
- ☐ Save and gameplay systems remain stable over long sessions.

Phase 7 — Closed testing and production readiness

Objective

Verify that the offline game is stable enough to be treated as a real product candidate.

Checklist

- ☐ Run compatibility and regression tests.
- ☐ Rehearse save corruption recovery.
- ☐ Run long-duration play sessions.
- ☐ Validate wipe and reset flow.
- ☐ Test balancing and progression pacing.
- ☐ Review privacy, localization, and accessibility requirements.
- ☐ Test Production build pipeline and release flow.

Exit criteria

- ☐ The game is stable enough for serious testing.
- ☐ Save, world, progression, and combat all behave predictably.
- ☐ Release process is known and repeatable.

Phase 8 — Launch stabilization

Objective

Ship a stable offline build, observe live issues, and fix them before adding online systems.

Checklist

- ☐ Prepare staging build and launch build.
- ☐ Turn on crash reporting and telemetry.
- ☐ Review early issue reports and patch flow.
- ☐ Track player pain points and balancing faults.
- ☐ Confirm rollback and emergency patch flow.

- ☐ Maintain ops runbooks and support workflow.

Exit criteria

- ☐ The offline build is stable in the field.
 - ☐ The team has a functioning issue and patch process.
 - ☐ The project is ready for the next phase: online and server architecture.
-

Phase 9 — Multiplayer/server enablement

Objective

Introduce the dedicated authoritative server only after the single-player loop is proven stable.

Checklist

- ☐ Define authoritative server ownership boundaries.
- ☐ Implement session management and gateway flow.
- ☐ Add player login and identity mapping.
- ☐ Move gameplay-critical systems to server authority.
- ☐ Add client prediction and reconciliation for movement.
- ☐ Add world streaming and synchronization rules.
- ☐ Validate inventory and combat replication.
- ☐ Add admin/moderation and server observability.

Exit criteria

- ☐ Multiplayer matches the single-player ruleset without breaking trust boundaries.
 - ☐ Server authority is enforced.
 - ☐ Gameplay state remains correct under real network latency.
-

Phase 10 — Live ops and platform maturity

Objective

Create a sustainable operational model for a live survival game.

Checklist

- ☐ Implement scheduled wipe or season lifecycle.
- ☐ Add content update pipeline and event-driven updates.
- ☐ Review telemetry and balance trends regularly.
- ☐ Build moderation workflow and reporting tools.
- ☐ Implement rollback and hotfix flow for production.
- ☐ Decide whether vehicles, deeper AI, or extra progression systems are part of the next milestone.

Exit criteria

- ☐ The live game has a repeatable operational loop.
 - ☐ Team responsibilities for ongoing content and support are defined.
 - ☐ The project has a clear roadmap beyond the initial release.
-

Offline-first gate before multiplayer work

Do not begin multiplayer/server architecture before these are true:

- ☐ Core survival loop is playable and stable offline.
- ☒ Save/load is versioned and reliable.
- ☐ Combat and building behavior are stable.
- ☐ Progression and resource loops are proven.
- ☐ Performance budgets are established and met.
- ☐ The game world can survive long sessions without corruption.
- ☐ The team has a stable debugging and telemetry workflow.

This is the main product risk gate for the entire project: if the offline game is unstable, the online version will be dramatically harder and more expensive to fix.

Unity development rule set

- ☐ Keep gameplay logic separate from UI logic.
 - ☐ Use a fixed simulation tick for world-critical behaviors.
 - ☐ Treat save data as versioned game data.
 - ☐ Do not trust client state for any gameplay-authoritative action.
 - ☐ Keep content data-driven and configurable.
 - ☐ Build debug and telemetry tools as first-class systems.
 - ☐ Maintain deterministic systems for world and progression logic.
-

3. Non-Negotiable Architectural Rules

These rules should become engineering standards rather than suggestions.

Rule 1 — The server owns truth

The client may request: movement, firing, interaction, pickup, crafting, building, inventory changes, container access, resource gathering.

The client must not be trusted to declare: damage dealt, item ownership, resource quantities, successful hits, building validity, crafting completion, inventory contents, progression unlocks, world-state changes.

Rule 2 — Simulation and presentation are separate

Server simulation should not depend on UI, rendering, animation state, particle systems, client-only physics effects, or frame rate. Client presentation consumes authoritative state and predicts where

appropriate. **This applies to sound as well as graphics** — gameplay-relevant audio (Section 12) is a server-authoritative event, not a client presentation detail.

Rule 3 — Every persistent schema is versioned

Never write an unversioned save structure to production. Each persisted object should contain a schema/version identifier and have a migration path.

Rule 4 — Every gameplay event is attributable

Important actions should have an actor, timestamp/tick, world/server identifier, and event type. Examples: player connected/disconnected, item created/destroyed/transferred, weapon fired, damage applied, building placed/destroyed, resource gathered, recipe crafted, admin action. **This rule applies to moderator and admin actions (Section 22) exactly as it applies to player actions** — a ban with no attributable record is as much a data-integrity failure as an untracked item transfer.

Rule 5 — Deterministic systems use controlled randomness

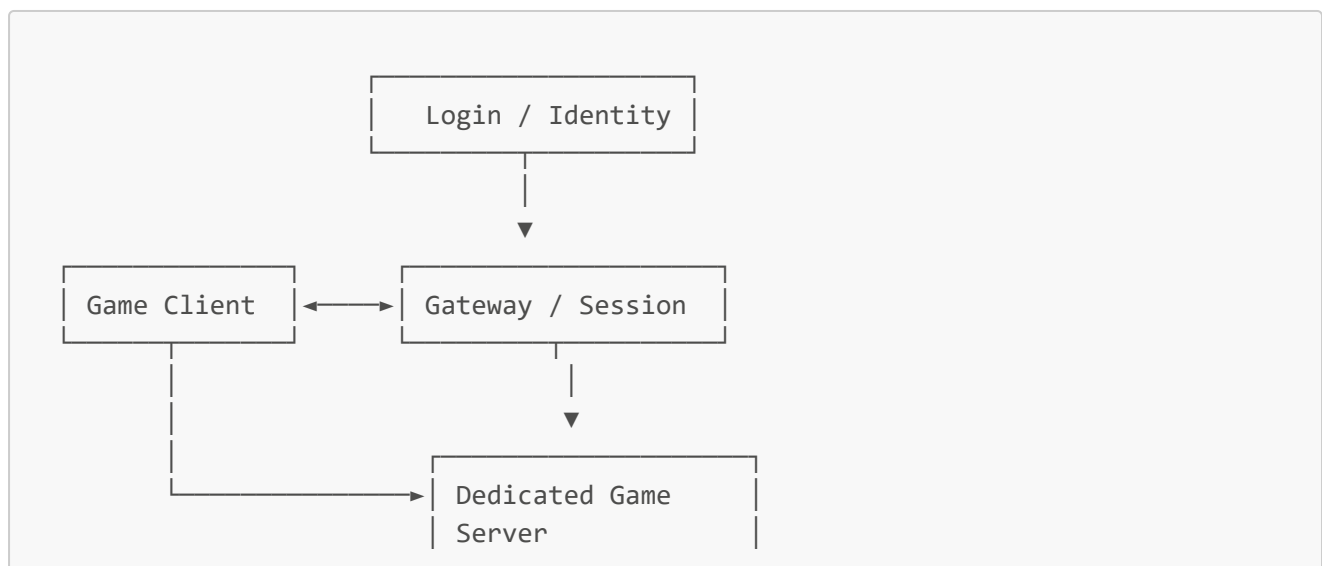
Do not call an uncontrolled random API from terrain, resource, monument, or simulation code. Determinism is a cross-system contract, enforced through code review, linting, and tests — not tribal knowledge.

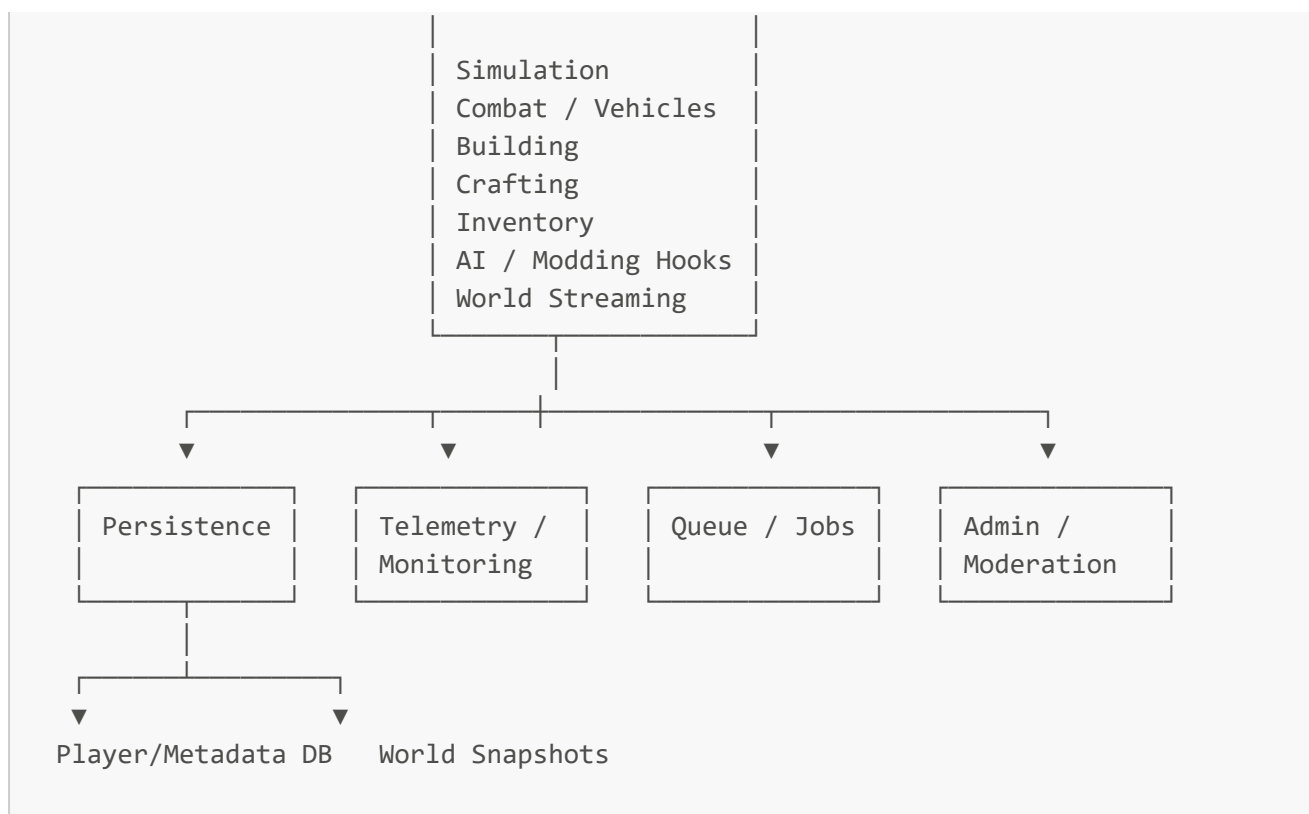
Rule 6 — Expensive work must have a budget

Every server system should have an explicit budget: CPU time per tick, memory, network bandwidth, database I/O, maximum work per frame/tick, maximum queue length. A feature is not production-ready simply because it is functionally correct. **This includes vehicle physics (Section 11) and sound-event processing (Section 12), which are easy to omit from initial budget planning because they weren't part of the original six-system scope.**

4. High-Level System Architecture

A practical production architecture is:





Recommended ownership boundaries

Client

Owns: rendering, UI, local animation, local audio (non-gameplay), input collection, client prediction, interpolation, presentation-only effects.

Game server

Owns: authoritative movement, world state, physics relevant to gameplay (including vehicles), inventory, crafting, combat, building, AI, resource state, player vitals (Section 8), gameplay-relevant sound events (Section 12), player state, validation, persistence scheduling.

Backend services

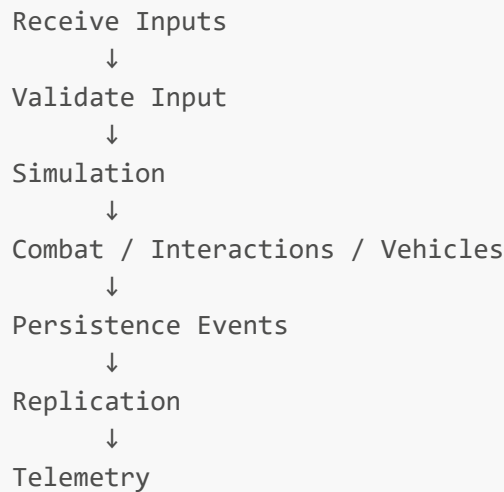
Own: authentication, account metadata, server discovery, player profiles, analytics, moderation case management, entitlement/platform data, operational tooling, localization content delivery.

Do not turn the game server into a general-purpose web backend. Keep gameplay latency-sensitive logic close to the simulation.

5. Networking and Server Simulation

All downstream systems depend on this contract.

5.1 Fixed simulation loop



Never make gameplay correctness depend on variable render-frame timing.

5.2 Input sequence numbers

Every client input should include at minimum: sequence number, client simulation timestamp, movement/input state, aim state, relevant action flags. The server acknowledges processed input sequences.

5.3 Client prediction

1. Capture input.
2. Apply local prediction.
3. Send input to server.
4. Receive authoritative state.
5. Reconcile.
6. Reapply unacknowledged inputs.

Prediction is only used for responsiveness; it does not transfer authority to the client. **Vehicle driver input follows the same prediction/reconciliation pattern (Section 11); passenger positions do not — they are server-parented, not independently predicted.**

5.4 Snapshot replication

Do not continuously replicate every entity to every player. Use spatial interest management, entity relevance, prioritized updates, delta compression, snapshot baselines, and lower update rates for distant entities.

5.5 Network validation

The server should reject: impossible movement, impossible fire rate, invalid inventory transitions, interaction outside range, invalid build placement, impossible resource collection, invalid timestamps, malformed packets, oversized requests, sequence abuse.

6. Building & Stability System

6.1 Structure data model

```
StructureNode
- entityId
- chunkId
- gridPosition
- rotation
- material
- health
- foundation/attachment type
- connections
- grounded state
- stability value
- dirty flag
- ownership/team id
- construction timestamp
- destruction state
```

Do not store redundant derived data unless it has a measured performance benefit.

6.2 Placement validation

1. Player has build permission.
2. Player owns required resources.
3. Item is unlocked.
4. Placement position is legal.
5. Rotation is legal.
6. Collision rules pass.
7. Socket/attachment rules pass.
8. Terrain rules pass.
9. Stability constraints pass.
10. The structure is committed atomically.

The client can preview placement, but the server decides whether placement succeeds.

6.3 Stability algorithm — incremental worklist model

1. Placement calculates local stability: `min(neighbor.stability) * decayFactor(material, connectionType)`.
2. Destruction marks neighboring nodes dirty, pushed to a worklist.
3. Dirty nodes recompute stability from still-valid neighbors; if changed, their own neighbors are marked dirty.
4. Propagation completes when the worklist is empty.
5. Nodes below the destruction threshold are queued — not destroyed inline.

6. Destruction executes as a separate pass, after propagation settles.

This separation avoids order-dependent collapse bugs.

6.4 Edge cases to test explicitly

Cycles, multiple foundations, disconnected structures, simultaneous destruction, support through different materials, diagonal connections, partial destruction, server restart during collapse, chunk boundaries, very large player-built bases, two players modifying the same structure in one tick, collapse while another player is interacting with the structure.

6.5 Performance protection

```
maxStabilityNodesPerTick  
maxCollapseEntitiesPerTick  
maxStructurePropagationTimeMs
```

If work exceeds budget: continue asynchronously where safe, spread work over ticks, or enter a controlled deferred state. Large raid-triggered components are a product/performance decision, not merely an algorithm choice.

7. Procedural World Generation

7.1 Generation pipeline

```
Master Seed  
  ↓  
Terrain Height  
  ↓  
Temperature / Moisture  
  ↓  
Biome  
  ↓  
Resources  
  ↓  
Rivers  
  ↓  
Roads  
  ↓  
Monuments  
  ↓  
Spawn Points  
  ↓  
Nav / AI Data  
  ↓  
Chunk Validation
```

Note: the temperature/moisture layer here seeds *biome classification*; it is distinct from the live, tick-updated player temperature system in Section 8, though the two share the biome data as an input.

7.2 Determinism contract

```
RandomSeed = Hash(  
    WorldSeed,  
    GeneratorVersion,  
    ChunkX,  
    ChunkZ,  
    LayerId  
)
```

This prevents changes in one generation layer from accidentally changing unrelated layers. Do not rely solely on `seed + offset`.

7.3 Generator versioning

A saved world must remember the generator version that produced it. If generator version 5 changes terrain rules, do not silently regenerate an old world using version 6. Choose explicitly between frozen generation for existing worlds, controlled world migration, or a full wipe (see Section 14 for wipe orchestration — a generator version bump is the correct, natural trigger point for a scheduled wipe).

7.4 Chunk lifecycle

```
Unloaded → Requested → Generating → Generated → Active → Inactive → Persisted →  
Unloaded
```

The server should never unload a chunk containing uncommitted persistent changes.

7.5 Resource generation

Use deterministic spawn points, biome weighting, density constraints, minimum spacing, respawn timers, server-side resource state. Avoid purely random per-frame spawning.

7.6 Monument placement

Score candidates on terrain flatness, biome compatibility, distance constraints, footprint validation, then apply terrain blending. Human tuning is required for visual falloff and placement feel — this is a design judgment AI-generated candidates can support but not replace.

7.7 Rivers and roads

Rivers: deterministic sources, downhill flow simulation, loop prevention, valid sinks, minimum/maximum length, intersection validation with monuments.

Roads: A* pathfinding, slope cost, biome cost, water cost, monument connectivity, road crossing rules.

7.8 World validation pass

After generation, automatically validate: unreachable spawn points, impossible terrain, overlapping monuments, invalid roads, river loops, missing resources, excessive resource density, missing navigation data, disconnected important locations. A generated world that looks valid is not necessarily a playable world.

8. Survival & Environmental Systems

This is the system that makes the game a *survival* game rather than a shooter with building — it is the core moment-to-moment pressure loop that the building, crafting, and combat systems exist to support, and it needs its own architectural treatment.

8.1 Player vitals model

```
PlayerVitals
- health
- hunger (0-100, decays over time, restored by food)
- thirst (0-100, decays over time, restored by water)
- temperature (computed, not stored – see 8.2)
- status effects: bleeding, radiation, poison, etc. (stacked, each with its own tick-damage and duration)
```

All vitals decay and apply damage **on the server tick**, under the same authority rule as every other system — never trust a client-reported hunger/thirst/health value.

Vitals do not need combat-tick precision. A lower-frequency vitals tick (e.g., 1-2 Hz) is usually sufficient and preserves CPU budget for combat-critical work.

8.2 Temperature as a computed value

Temperature should not be stored server state that drifts independently — it should be recalculated from:

```
temperature = f(biome, weather, timeOfDay, nearbyHeatSources,
clothingInsulation)
```

Recomputing it each vitals tick avoids an entire class of desync/staleness bugs where stored temperature falls out of sync with the conditions that should determine it.

8.3 Weather system

- Server-authoritative weather state machine per region/world (e.g., clear → overcast → rain → storm).
- Broadcast to clients for VFX/audio, but weather must also feed directly into temperature (8.2) and visibility calculations — it needs to be **gameplay-authoritative**, not purely cosmetic, or the system is decorative and the engineering investment isn't justified.
- Weather transitions should themselves be deterministic-seedable per world/session where reproducibility matters (e.g., for debugging a reported issue).

8.4 Day/night cycle

A server-tracked time-of-day float, replicated to clients. Feeds: visibility, AI perception radius (Section 16.2), and temperature (8.2). Keep this as a single source of truth rather than letting client and server clocks drift independently.

8.5 Status effects framework

Build a generic status-effect stack (bleeding, radiation, poison, and future additions) rather than one-off hardcoded systems per effect:

```
StatusEffect
- effectType
- magnitude
- remainingDuration
- tickDamage / tickModifier
- stacking rule (refresh, stack, ignore)
```

This is a good AI-assisted implementation target once the stacking rules are specified by a designer — the pattern is mechanical, the design decisions (does a second radiation exposure refresh or stack the timer?) are not.

8.6 Performance protection

Vitals and status-effect processing scale with player count, not entity count — budget it explicitly (Section 6.5's pattern applied here): `maxVitalsProcessedPerTick`, with graceful degradation (lower tick frequency under load) rather than a hard failure if the budget is exceeded.

9. Crafting, Inventory & Progression

9.1 Item definition

```
ItemDef
- id
- version
```

- category
- maxStack
- weight
- durability
- recipe
- outputQuantity
- craftTime
- workbenchTier
- unlockId
- rarity
- metadataRules

9.2 Item instance

Separate the definition from the runtime instance:

```
ItemInstance
- instanceId
- itemDefId
- quantity
- durability
- customData
```

This distinction is essential for weapons, tools, skins, containers, and unique items.

9.3 Inventory transaction model

Inventory changes should be atomic:

```
BEGIN TRANSACTION
Validate player owns ingredients
Validate recipe
Remove ingredients
Create output
Update inventory
Emit item-created/item-consumed events
COMMIT
```

Never perform `remove wood` → `wait` → `craft` → `add rifle` as unrelated operations — a server crash between them can duplicate or destroy items.

9.4 Crafting

Server validates: recipe exists, recipe version, unlock, workbench, proximity, ingredients, inventory capacity, crafting queue capacity. Craft completion must be server-timed — never trust the client to report that the craft timer finished.

9.5 Tech tree

A `HashSet<itemId>` per player is appropriate when unlock relationships are simple. Introduce explicit prerequisite-graph data only when requirements justify it — resist adding structure the requirements don't yet need.

9.6 Economy tooling

Automated reports for: total raw cost, intermediate cost, crafting time, resource-per-minute assumptions, time-to-tier progression, item sink/source balance. Unrolling recipes into total raw-resource cost should be a formal, repeatable balancing tool, not a one-off spreadsheet exercise.

10. Combat & Ballistics

10.1 Weapon data

```
WeaponDef
- id
- fireMode
- fireRate
- magazineSize
- reloadTime
- projectileType
- muzzleVelocity
- gravityScale
- drag
- spread
- recoilProfile
- damageProfile
- penetrationProfile
- range
- durability
```

10.2 Fire validation

Server checks: weapon equipped, weapon belongs to player, fire cooldown, ammunition, reload state, player alive state, valid aim data, valid timestamp, plausible firing origin.

10.3 Lag compensation

Maintain a bounded historical hitbox buffer (starting point: 300-500 ms window, tuned to actual network conditions and gameplay requirements). The server must clamp client timestamps to a permitted rewind window — never allow a client to request arbitrary historical state.

10.4 Hitscan resolution

1. Validate shot.
2. Rewind target hitboxes (never the shooter's own).
3. Perform raycast.
4. Resolve penetration.
5. Calculate damage.
6. Apply damage to current authoritative entity state.
7. Emit combat event.
8. Replicate result.

10.5 Projectiles

Decide explicitly whether simulation is fully server-simulated, client-visual-with-server-authority, or a deterministic projectile model. Never use client physics as the authority for damage.

10.6 Penetration

```
DamageType × MaterialType → penetration depth → damage multiplier → penetration loss
```

A data-driven lookup matrix — data-entry complexity, not algorithmic complexity.

10.7 Anti-cheat signals specific to combat

Impossible fire rates, impossible movement, impossible aim transitions, abnormal hit ratios, impossible recoil patterns, repeated invalid actions, packet manipulation, timestamp abuse. Do not auto-ban from one weak signal — use layered evidence (Section 22's detection pipeline).

11. Vehicles & Dynamic Physics Entities

Cars, boats, and similar player-driven physics entities introduce a distinct networking problem: **multi-passenger authoritative physics bodies**, not single-character prediction.

11.1 Vehicle data model

```
VehicleEntity
- physicsBody (server-simulated rigidbody)
- seats[] (driver + passenger slots, each bound to a playerId or empty)
- fuel/durability
- ownership/lock state
- inputState (from driver only – passengers cannot submit control input)
```

11.2 Authority model

- The server simulates the vehicle's physics.
- The **driver's client predicts locally** using the same prediction/reconciliation pattern as player movement (Section 5.3), then reconciles against server state.
- Other clients (including passengers) interpolate the replicated position — passengers are **positionally parented** to the vehicle on the server, not predicted independently. This avoids desync bugs where a passenger's own movement prediction fights the vehicle's.

11.3 Combat interaction

Vehicles participate in the damage/penetration model from Section 10.6 — a vehicle hull is effectively a material type in the penetration matrix. Destruction should feed the same attributable-event rule (Rule 4) as building destruction.

11.4 Performance budget

Vehicle physics bodies are a meaningfully different CPU cost than character movement. Give them their own line in the tick budget (Section 6.5's pattern): `maxActiveVehiclePhysicsBodies`, load-tested independently rather than assumed to fit inside the general physics budget.

12. Audio & Sound Propagation

In a genre where footstep and gunshot audio are gameplay information (stealth, positional awareness), sound is a **networked gameplay event**, not a purely client-side presentation concern.

12.1 Sound event model

```
SoundEvent
- sourcePosition
- soundType (footstep, gunfire, explosion, door, etc.)
- radius
- emittedTick
```

Gameplay-relevant sounds are emitted as server-authoritative events with a position and radius, replicated only to clients within range — architecturally the same interest-management pattern as entity replication (Section 13.5), applied to a different data type.

12.2 Cosmetic audio

Ambient audio (wind, music, UI sound) stays fully client-side and never touches the network — do not route non-gameplay audio through the sound-event system, it adds replication cost for no gameplay value.

12.3 Occlusion

Explicitly decide whether sound occludes through geometry (walls muffling/blocking sound). If yes, this needs its own lightweight server-side line-of-sound check — do not silently reuse the AI perception line-of-sight raycast (Section 16.2) without confirming the cost is acceptable, since footstep-event frequency is much higher than AI perception-check frequency.

12.4 Accessibility tie-in

Any sound that carries gameplay information needs a non-audio channel available for players who need it (directional visual indicator, subtitle/caption) — see Section 19.3.

13. Player, Inventory and World Persistence

Persistence is more than serialization.

13.1 Persistence domains

Player persistent state: identity, inventory, equipment, progression, stats, preferences, position, health/status where appropriate.

World persistent state: structures, containers, resource changes, monument state, doors/locks, traps, dropped items, terrain deltas, vehicle state.

Operational state: server metadata, world version, last save, snapshot version, active players, server health.

Keeping player state cleanly separated from world state (rather than commingled) is also what makes data-privacy export/deletion requests (Section 23) technically tractable.

13.2 Snapshot + event strategy

Periodic Snapshot + Short-lived Event Journal

Snapshots prevent replaying an enormous history after restart. The event journal protects against losing recent changes between snapshots.

13.3 Save lifecycle

Simulation → Dirty State → Snapshot Request → Consistent Snapshot →
Serialize → Compress → Persist → Checksum → Commit

13.4 Atomic writes

Never overwrite the only valid world snapshot:

```
world.snapshot.previous  
world.snapshot.current  
world.snapshot.temp
```

Write to temp → flush/complete → verify checksum → atomically promote.

13.5 Schema versioning

```
CurrentVersion  
Migration(CurrentVersion → NewVersion)
```

Migration tests must include old production-style data.

13.6 Sparse terrain deltas

Store only deviations from deterministic seed-generated terrain. Add: chunk-level checksums, delta compression, maximum delta count, corruption detection, fallback to previous snapshot.

13.7 Save consistency invariant

A snapshot represents one coherent simulation state, never a mixture of two ticks.

Copy-on-write/double-buffer snapshotting is an area requiring heavy human review regardless of how it was drafted.

14. Wipe & Season Lifecycle Management

Rust-likes commonly run on a **recurring wipe cycle** — this is a first-class operational concept that interacts directly with persistence (Section 13), world generation versioning (Section 7.3), and economy tooling (Section 9.6).

14.1 Wipe types

```
WipeType  
- map wipe (new world generated; structures/resources reset; player progression  
  may persist)  
- full wipe (map + player progression + inventory reset)  
- forced wipe (triggered by a breaking generator/schema version change)
```

14.2 Wipe orchestration

A wipe is a scheduled, server-orchestrated operation, not a manual snapshot deletion:

```
Announce wipe window
↓
Drain players
↓
Final snapshot/archive of the old world (rollback + player-facing stats)
↓
Generate new world under the new generator version
↓
Reset player state per wipe type
↓
Reopen
```

14.3 Generator version alignment

Schedule breaking generation changes (Section 7.3) to land on wipes rather than mid-season, so "same seed + version reproduces the same world" never has to be violated on a live world.

14.4 Testing

A full wipe cycle should be executed against a test population at least once before it is trusted on a live world — treat it as a first-class operation with its own soak/failure tests (Section 29), not an assumed-safe maintenance script.

15. Performance & Scaling

15.1 Performance budget

Define budgets before optimization:

Area	Initial target
Simulation	< fixed tick budget
Networking	bounded outgoing bandwidth/player
Persistence	no blocking of main simulation
AI	bounded CPU per tick
Physics (character + vehicle)	bounded active-body count
Vitals/status effects	bounded per-tick processing
Sound events	bounded emissions per tick
Memory	explicit server memory ceiling
Generation	asynchronous/chunk-budgeted

Actual numbers should be determined from target hardware and player count.

15.2 Object pooling

Pool: projectiles, loot, corpses, temporary effects, AI query objects, network message buffers, vehicle physics bodies where reuse is safe. Do not pool everything automatically — pool high-frequency allocations after profiling.

15.3 Garbage collection

Measure allocations in: server tick, combat, AI, networking, inventory operations, chunk streaming, vitals processing. Zero-allocation goals are useful for hot paths but should not justify unreadable code in cold paths.

15.4 Simulation LOD

```
Tier 0 – Full simulation
Tier 1 – Reduced frequency
Tier 2 – Simplified simulation
Tier 3 – Dormant
```

Transition rules based on: player proximity, combat state, ownership, recent activity, entity type. Never put a player-visible combat entity into dormant simulation simply because it is temporarily far from one player.

15.5 Interest management

Start with a uniform spatial grid (`CellX`, `CellZ` per entity). Clients query neighboring cells based on relevance radius. Upgrade to a quadtree only if profiling proves the grid insufficient. Sound events (Section 12.1) use this same relevance mechanism.

15.6 Churn handling

Players crossing cell boundaries can create excessive subscriptions. Use cached cell membership, hysteresis around relevance boundaries, batched subscription updates, dirty relevance sets, priority-based replication.

16. AI / NPC Architecture

16.1 NPC state separation

```
Perception → Decision → Navigation → Action → Animation
```

Do not build a giant monolithic AI controller.

16.2 Perception

Server-authoritative queries: distance, line of sight, hearing/noise (feeds from Section 12's sound events), damage events, target visibility, threat score. **Perception radius is affected by day/night cycle and weather (Section 8.3-8.4)** — treat this as an explicit input, not an emergent side effect.

16.3 Decision system

A utility system, behavior tree, or state machine can work — choose the simplest model that supports required behaviors:

```
Idle → Investigate → Acquire Target → Chase → Attack → Lose Target → Search → Return
```

16.4 Navigation

Navigation data generated per world/chunk and versioned. Validate navigation after procedural world generation (Section 7.8).

17. Modding & Plugin Architecture

For this genre specifically, third-party server modding is close to a business-critical retention system — a major reason player-run server populations in comparable games have stayed alive for years — not an optional extra.

17.1 The core decision

Decide early whether the game exposes a **server-side scripting/plugin API** (Lua, C# assembly loading, or a sandboxed script host) letting community server operators customize rules, economy, and events without your studio shipping code changes. This is architecture-shaping, not a late add-on.

17.2 Hook surface

If modding is in scope, every gameplay system that should be moddable needs an explicit **hook/event surface** designed in from the start:

```
OnCraftRequested / OnCraftCompleted  
OnDamageApplied  
OnStructurePlaced / OnStructureDestroyed  
OnPlayerConnect / OnPlayerDisconnect  
OnChatMessage  
OnEconomyTransaction
```

Retrofitting mod hooks onto systems built without them is far more expensive than designing them in.

17.3 Trust boundary and safety

Plugin code runs on operator-controlled servers, not yours — a lower-trust-boundary problem than your own backend. Plugin APIs that can corrupt persistence format or bypass the atomic-transaction rules (Section 9.3) need explicit guardrails: plugins call validated, sandboxed APIs; they never get raw write access to inventory or persistence state.

17.4 Rollout model

Treat the modding API as its own project with its own phased rollout: internal dogfood → limited community beta → public, with its own documentation and community support commitment (see Phase 10, Section 32).

18. Interaction System

Survival games contain many interactions: doors, storage, switches, crafting stations, resource nodes, vehicles, traps, locks, workbenches.

Standardize them through an interaction contract:

```
CanInteract(player, entity)
ValidateInteraction(request)
ExecuteInteraction(request)
EmitInteractionEvent()
```

This avoids each gameplay system inventing its own authority checks.

19. Onboarding, Localization & Accessibility

Retention depends as much on how a new player learns the game and whether it reaches them at all as it does on any server system.

19.1 Onboarding

A first-session experience (tutorial island, guided objectives, or contextual hints) is a product decision, but the engineering requirement is that onboarding state is **server-tracked per player** (has completed tutorial, current guided step) using the same persistence/versioning rules as everything else (Section 13.5) — not a separate, unversioned client-only flag.

19.2 Localization

Externalize all player-facing strings (item names, UI, tutorial text) into a key-based localization table from day one. Retrofitting localization onto hardcoded strings across hundreds of item definitions is expensive and error-prone; this is cheap to get right early and expensive to fix late.

19.3 Accessibility

Colorblind-safe UI/status indicators, remappable controls, and subtitle/caption support for any voiced or audio-cue-dependent gameplay information — directly tied to Section 12.4, since if sound carries gameplay information, some of that information needs a non-audio channel too.

20. Ownership, Teams and Permissions

Building and storage require explicit authorization. Support: player ownership, team ownership, shared access, locked access, administrator access.

Every protected interaction should answer:

Who owns it? Who can access it? Why are they allowed? When was permission granted?

Do not encode permissions solely in client UI.

21. Anti-Cheat and Abuse Prevention

Design from the first network prototype, not added after launch.

21.1 Server-side validation

Movement, acceleration, jump/fall limits, fire rate, reload, inventory, crafting, resource gathering, building, interaction distance, damage, projectile timing, vehicle input plausibility.

21.2 Rate limiting

Packets, RPC/action requests, inventory operations, interaction requests, crafting requests, chat, server queries.

21.3 Detection pipeline

Signal → Score → Correlate → Review / Automated Action → Audit Record

Prefer multiple independent signals over a single heuristic.

22. Server Administration & Moderation Tooling

Section 21 covers *detecting* bad behavior; this section covers the tooling humans need to *act* on it.

22.1 Admin panel

Player lookup, teleport/spectate for investigation, kick/ban/mute with duration and reason, inventory/build inspection, server config hot-reload where safe.

22.2 Player reporting flow

In-client report → attaches recent server-side event log (Rule 4 makes this straightforward, since the data already exists) → queued for moderator review.

22.3 Moderator accountability

Every moderator action is itself an attributable event (Rule 4 applies here exactly as it does to player actions) — who banned whom, when, why, reversible or not.

22.4 Ownership model decision

Decide early whether moderation is studio-run, community-server-operator-run, or both — a community-operator model needs a much more complete self-service admin panel shipped to every server owner, not an internal-only tool. This decision also interacts directly with the modding rollout model (Section 17.4).

23. Security

23.1 Client security

Assume the client can be modified. Never store secrets in client code, engine assets, configuration files shipped to players, or network messages.

23.2 Backend security

Short-lived credentials, server-to-server authentication, least-privilege access, encrypted transport, secrets management, key rotation.

23.3 Persistence security

Protect against: duplicated item IDs, replayed transactions, invalid version migrations, partial writes, unauthorized player access, cross-server state races.

24. Legal, Compliance & Data Privacy

Regulatory obligations have real engineering consequences, not just legal ones.

24.1 Data privacy

Player data deletion/export requests need a defined technical path — this is far easier when player data is already cleanly separated into its own persistence domain (Section 13.1) rather than scattered across ad hoc tables.

24.2 Age rating and content compliance

Violence, chat, and voice features may need region-specific gating. Build feature flags for this early rather than assuming one global ruleset.

24.3 Chat/voice moderation

Ties back into Section 22's moderation tooling — plan for it as one system, not two.

25. Concurrency and Distributed Server Issues

If multiple server processes can touch the same player/world state, define ownership explicitly.

25.1 Player session ownership

A player should normally have one authoritative gameplay session.

Old Session → Session Revocation → New Session

Avoid two active servers mutating the same inventory.

25.2 Distributed locking

Use locks only where required. Prefer ownership/partitioning over frequent distributed locks.

25.3 World ownership

A world shard should have a single simulation authority. On crash: mark server unhealthy → prevent new connections → recover latest valid snapshot → replay safe journal events → validate state → resume or provision replacement server.

26. Server Lifecycle and Operations

Dedicated servers need explicit states:

Provisioning → Starting → Loading World → Ready → Running →
Draining → Saving → Stopped → Unhealthy → Recovering

26.1 Graceful shutdown

1. Stop new connections.
2. Notify players.
3. Stop new persistent transactions.
4. Complete/flush safe operations.
5. Create final snapshot.
6. Verify persistence.
7. Close connections.
8. Report clean shutdown.

27. Observability

27.1 Metrics

Server tick duration, tick overruns, player count, network bandwidth, packet loss, latency, entity count, active chunks, memory, GC allocations, persistence duration, save failures, stability propagation size, combat queries, vehicle physics load, vitals-processing load, AI CPU, database latency, queue depth.

27.2 Structured logs

```
timestamp, serverId, worldId, playerId, entityId, tick, operation, errorCode,  
correlationId
```

Never log secrets or sensitive credentials.

27.3 Alerts

Repeated save failure, tick budget violations, memory growth, server crash loops, database failures, abnormal player disconnect rates, replication saturation.

28. Analytics and Gameplay Telemetry

Track enough information to answer: Where do players die? Where do they quit? Which resources are scarce? Which weapons dominate? How long does progression take? Which recipes are ignored? Which monuments are visited? Which server regions have poor latency? What actions correlate with crashes?

Do not collect data merely because it is technically possible. Define a telemetry schema and retention policy. **This data should explicitly feed the live-ops patch-planning process (Section 32, Phase 9)** — define who reviews it and how often, not just that it's dashboarded.

29. Testing Strategy

AI-generated reference implementations and property-based tests should be a formal testing layer, not an ad hoc practice.

29.1 Unit tests

Recipe validation, inventory transactions, stability calculation, damage calculation, penetration, permission checks, procedural seed functions, schema migrations, vitals decay curves.

29.2 Property-based tests

Especially valuable for structure graphs, inventory conservation, deterministic generation, serialization round trips, recipe graphs, movement validation. Example invariant:

```
Items before transaction + Items created - Items destroyed = Items after transaction
```

29.3 Integration tests

Client ↔ server, persistence, chunk loading, combat + lag compensation, inventory + crafting, building + stability, vehicle physics + passenger sync, sound-event relevance.

29.4 Soak tests

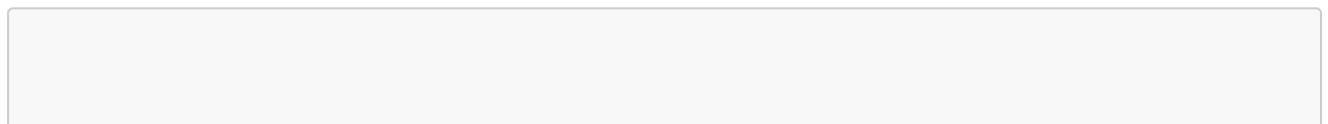
Simulated servers with hundreds of bots, large bases, high projectile counts, frequent chunk transitions, repeated reconnects, save/load cycles, **a full wipe cycle (Section 14.4)**.

29.5 Failure tests

Server crash during save, database unavailable, network packet loss, duplicated request, delayed packet, corrupted snapshot, invalid client timestamp, extremely large structure, malformed client packet.

30. CI/CD and AI-Assisted Engineering Workflow

30.1 Branch pipeline



30.2 Good AI tasks

Boilerplate, test generation, serializers, data conversion, editor tools, documentation, repetitive definitions (item data, localization key scaffolding), reference implementations, profiling helpers, migration scaffolding.

30.3 Human-owned tasks

Network authority model, tick architecture, persistence consistency, lag compensation, vehicle passenger-sync correctness, anti-cheat policy, security, concurrency, scalability thresholds, schema contracts, performance budgets, modding trust-boundary design.

30.4 AI review rule

No AI-generated code should be merged without a human reviewer understanding: what the code does; why it is correct; what assumptions it makes; what happens under failure; what happens at scale; what state it mutates; whether it introduces nondeterminism.

This principle — **AI drafts, humans own correctness-critical algorithms** — is a project governance rule (Section 37), not a coding-style preference.

31. Client Build, Patch & Distribution Pipeline

Section 30 covers CI for the codebase; this covers how a finished build reaches players.

31.1 Asset strategy

Decide what ships in the base client vs. downloads on demand — affects both initial install size and patch size for content updates.

31.2 Staged rollout

New builds go to a small percentage of servers/players first, with automated health-metric comparison (Section 27) against the previous version, before full rollout — catches performance regressions before they hit the whole population.

31.3 Hotfix path

A separate, faster pipeline than the full release process for critical fixes (exploit patches, crash fixes). Should skip non-essential QA gates but never skip the determinism/persistence-compatibility tests (Section 29) — a bad hotfix that breaks save compatibility is worse than the bug it fixed.

31.4 Platform certification

Track console/storefront certification lead time explicitly against the phase roadmap (Section 32) — this is a fixed external lead time that doesn't compress under schedule pressure.

32. Complete Phased Roadmap

Estimates assume a **small senior team (4-8 engineers, ~1-2 designers, part-time art/audio support)**. These are planning-order-of-magnitude estimates, not commitments — validate against actual team composition before using them for external commitments.

Build the dependency chain first (this order prevents building gameplay on unstable contracts):

```
Offline Foundation → Core Survival Loop → Building → Combat → Procedural World  
→  
Progression and Content → Scale and Hardening → Closed Test → Launch  
Stabilization →  
Authoritative Networking → Live-Ops → Platform Maturity
```

Phase 0 — Technical Foundation

Deliver: Unity boot flow, modular MonoBehaviour boundaries with plain C# domain logic, fixed-timestep configuration, versioned local save/load with backup recovery, deterministic RNG, structured logging and debug tools, localization key infrastructure, player-data domain separation, a local test scene, and basic CI for editor tests and builds.

Team: 3-4 senior engineers. **Duration:** 4-6 weeks.

Exit criterion: the game boots cleanly in the editor; repeated save/load cycles preserve valid state and recover from an interrupted write; schema versions are enforced; RNG repeatability tests pass; and logs and editor tests are usable. Multiplayer is gated on the offline-first exit criteria in Section 2A.

Phase 1 — Core Survival Loop

Deliver: player movement, gathering, inventory, item definitions, basic crafting, health, death/respawn, basic world chunks, hunger/thirst/temperature vitals, basic day/night cycle.

Team: 5-7 engineers + 1 designer. **Duration:** 6-10 weeks.

Exit criterion: player can gather → craft → use items → die → reconnect, with vitals decaying under server authority.

Phase 2 — Building

Deliver: placement, permissions, structures, stability, destruction, persistence, large-base performance tests.

Team: 4-5 engineers. **Duration:** 5-7 weeks.

Exit criterion: structures survive restart and large collapse events stay within server budget.

Phase 3 — Combat

Deliver: weapons, damage, projectiles/hitscan, lag compensation, death, loot, combat telemetry, gunfire/footstep sound propagation.

Team: 4-6 engineers. **Duration:** 6-9 weeks (lag compensation correctness work tends to run long — budget explicitly).

Exit criterion: combat remains authoritative and stable under simulated latency/loss.

Phase 4 — Procedural World

Deliver: seeded generation, biomes, resources, monuments, rivers, roads, navigation, chunk streaming, weather system.

Team: 3-5 engineers + 1 technical designer. **Duration:** 6-8 weeks.

Exit criterion: same seed + generator version produces the same world.

Phase 5 — Progression and Content

Deliver: workbenches, tech tree, expanded recipes, progression, NPCs, additional monuments, environmental systems, onboarding/tutorial flow. Basic vehicles if in scope for launch — otherwise explicitly defer to Phase 9.

Team: 5-7 engineers + 2 designers. **Duration:** 8-12 weeks (largest content surface area of any phase).

Phase 6 — Scale and Hardening

Deliver: load testing, server orchestration, autoscaling, crash recovery, backup/recovery, anti-cheat, observability, admin/moderation tooling, wipe orchestration.

Team: 4-6 engineers + 1 ops/SRE-focused engineer. **Duration:** 6-10 weeks.

Phase 7 — Closed Test / Production Readiness

Deliver: concurrency tests, long-duration soak tests, persistence corruption tests, security review, exploit testing, economy review, player telemetry review, completed localization pass, legal/compliance review, staged-rollout pipeline rehearsal.

Team: Full team + QA. **Duration:** 4-8 weeks.

Exit criterion: a data-privacy export/deletion request can be fulfilled end-to-end using real test-player data, and a full wipe cycle has been executed at least once against the closed-test population without data loss.

Phase 8 — Launch and Stabilization

Deliver: live monitoring war-room process for the first 1-2 weeks post-launch, first hotfix pipeline execution under real conditions, player-report → moderation-action loop running end-to-end at real population scale.

Team: Full team on-call rotation. **Duration:** 2-4 weeks of heightened response, tapering.

Exit criterion: server stability metrics (tick budget, crash rate, save failure rate) hold within Section 15.1 budgets at real launch population for a full week without manual intervention.

Phase 9 — Live-Ops Cadence Established

Deliver: first scheduled wipe executed on the recurring cadence, fully automated per Section 14.2; first live-triggered event using a remote-config/event system; vehicles, if deferred from Phase 5; first full patch-planning cycle informed by telemetry review.

Team: Steady-state team, likely smaller than peak dev team, plus a dedicated live-ops/community role.

Duration: ongoing; first full cycle ~4-8 weeks to validate the process itself.

Phase 10 — Platform Maturity

Deliver: modding/plugin API, if committed to — treated as its own multi-quarter project with its own internal-dogfood → limited-beta → public rollout, not a single deliverable; cross-region/server-browser scaling if population growth requires it; accessibility feature completeness pass beyond MVP.

Team: Varies significantly by scope — a modding API alone can be a multi-quarter effort for 2-3 engineers plus documentation/community support.

Duration: an ongoing roadmap track, not a bounded phase — reprioritized each cycle against live telemetry and community feedback.

32.1 Required decision lock gates before implementation

The following decisions are not optional planning preferences. They are required project gates and must be formally approved before any phase proceeds beyond its first milestone. If a decision is open, the corresponding technical risk remains open and must be treated as active project risk until resolved.

Engine and architecture gate

- Finalize the engine, rendering pipeline, and simulation model.
- Project direction selected: Unity with modular MonoBehaviours and plain C# domain logic; adopt ECS only if profiling demonstrates a concrete need.
- Commit to the authoritative simulation model and the degree of client prediction allowed.
- Document the exact reason for the choice: gameplay complexity, hiring, toolchain, physics, networking, content authoring, or runtime performance.

Server model gate

- Decide whether production is a dedicated authoritative server-only model or a hybrid local-authoritative model for offline testing.
- Define whether there will be a dedicated world server process, shard architecture, or single-instance world for the first playable build.
- Confirm the target tick rate and the valid range for tuning.
- Define the maximum supported player concurrency per world for Phase 1-3, Phase 4-6, and launch.

Persistence gate

- Finalize the persistence format and schema versioning strategy.
- Decide exact storage boundaries: player data, world state, operational metadata, and snapshots.
- Define the path for upgrades between schema versions.
- Decide the save cadence, snapshot retention policy, and corruption recovery behavior.

World generation gate

- Decide the seed generation rules and whether the world is deterministic across server restarts.
- Lock the generator versioning strategy before any world is created.
- Define what counts as a valid world after generation and what defects force a regeneration or wipe.
- Commit to chunk streaming behavior, world boundaries, and large-world limits.

Economy and progression gate

- Define the final progression philosophy: survival loop first, progression second, or economy-first design.
- Lock the recipe graph model and the source of truth for crafting unlocks.
- Decide whether progression uses a simple unlock set, graph-based prerequisites, or a more complex gating model.
- Document how economy balancing is measured and tested.

Combat gate

- Decide the authoritative combat model: hitscan, projectile, hybrid, or deterministic event-driven simulation.
- Define the hitbox model, range validation, and fixed lag compensation policy.
- Lock the damage and penetration matrix.
- Define the anti-cheat signal thresholds and review procedure.

Building and stability gate

- Finalize the build permission model: player-owned, team-owned, shared-access, or server-managed.
- Decide the exact placement validation pipeline and collapse propagation algorithm.
- Define the size limits for structures, foundations, and large collapse events.
- Establish the max-per-tick budget for propagation and destruction processing.

Social and moderation gate

- Decide whether moderation is studio-run, operator-run, or both.
- Finalize how players report misconduct and how moderator actions are recorded.
- Decide how admin commands are permissioned and which actions are restricted or logged.
- Lock the auditable event model for moderation, service actions, and player bans.

Modding gate

- Decide in writing whether modding is in-scope or out-of-scope.
- If in-scope, define the trust boundary, plugin API surface, and safety model.
- If out-of-scope, document this as a product decision so it does not become an accidental unplanned dependency later.

Distribution and launch gate

- Finalize the release pipeline strategy.
- Decide what is included in the base client, what is downloaded dynamically, and what is server-side.

- Define the staged rollout process for builds and updates.
- Define the hotfix path, rollback criteria, and post-launch monitoring requirements.

These gates are the baseline contract for every phase. If a gate is unresolved, the project is not ready to proceed to the next phase.

32.2 System ownership and responsibility matrix

This matrix defines who owns what and where the actual authority resides. Ambiguity is a production risk and must be resolved before implementation begins.

Area	Client owns	Server owns	Backend owns	Rationale
Input collection	Raw input, camera, UI input state	Input validation, authoritative movement, actions	Session identity and auth	Prevents client authority exploits
Player movement	Prediction, interpolation, animation	Final position, speed, collision, physics	Session metadata	Movement must be server-authoritative
Combat	Weapon visuals and aim feedback	Damage application, hit resolution, ammo checks	Metrics and anti-cheat telemetry	Combat correctness is critical
Building	Placement preview and feedback	Placement validation, permission, stability, destruction	Ownership and permission metadata	Server decides final validity
Inventory	Local display, drag UI	Atomic transaction validation and state mutation	Account metadata and profile data	Prevents duplicate or lost items
Crafting	UI timers and visuals	Recipe validation, timing, completion, output generation	Unlock/status metadata	Craft completion must be server-timed
Vitals	Local HUD and effects	Hunger/thirst/temperature logic and damage application	Health/performance telemetry	Survival loop is a real simulation
Weather	VFX and ambient visuals	weather state machine, gameplay impact	Regional metrics and event telemetry	Weather influences survival, not only visuals

Area	Client owns	Server owns	Backend owns	Rationale
Vehicles	Driver input prediction, local vehicle visuals	Physics simulation, seat authority, passenger state	Vehicle ownership data	Vehicles need authoritative multi-passenger handling
Audio	Local music, UI, ambient audio	Gameplay sound event generation and range checks	Analytics for sound usage	Gameplay sound is not merely cosmetic
World generation	Visual map preview, seed view	Seeded terrain generation, chunk state, validation	World metadata and versioning	Determinism and reproducibility matter
Persistence	Local save UI, load states	Snapshot generation, schema migration, atomic writes	Backup/restore workflows	Save integrity is a server-side concern
Admin tools	Local UI only	Commands, logs, event attribution, enforcement	Moderation workflow and audit storage	Human action must be attributable
Analytics	UI instrumentation and local playback telemetry	gameplay event emission and telemetry tagging	Long-term data warehouse/retention	Metrics drive operations and balancing

Authority rules

1. The server owns all state that can change game outcomes.
2. The client may render and predict, but not decide the truth.
3. The backend owns external identity, account metadata, and long-lived operational state.
4. Any gameplay-affecting action must be traceable to a validated server event.

32.3 Detailed implementation contracts

The following contracts are included to reduce ambiguity and remove the need for guesswork during implementation. They are the technical minimum that must exist before the game can be considered production-ready.

1. Player session contract

```
PlayerSession
- sessionId
- playerId
- serverId
- worldId
- authTokenId
- connectionState
- inputSequenceStart
- lastHeartbeatTick
- lastKnownPosition
- lastKnownRotation
- sessionVersion
- sessionCreatedAt
- sessionClosedAt
```

Rules:

- One active authoritative session per player per world.
- Old sessions must be revoked before a new session can claim ownership.
- Reconnects must preserve save state if allowed by server policy.
- Session logs must include reason codes for disconnects, kicks, and forced transfers.

2. Inventory transaction contract

```
InventoryTransaction
- transactionId
- playerId
- sourceContainerId
- targetContainerId
- itemInstanceId
- itemDefId
- quantityDelta
- reasonCode
- tick
- serverId
- validationHash
```

Rules:

- Never mutate inventory in multiple unsafely ordered steps.
- Validate before mutation.
- Always write an event record and update a checksum or ledger.
- The sum of inventory changes must be equal to the net change recorded in the transaction ledger.
- Invalid or interrupted transactions must roll back or remain in a safe failed state.

3. Building placement contract

```
StructurePlacementRequest
```

- placementId
- playerId
- structureDefId
- worldPosition
- rotation
- placementMode
- currentTick
- validationNonce

Server pipeline:

1. Validate player permissions.
2. Validate structure unlocks.
3. Validate placement legality.
4. Validate terrain and collision state.
5. Validate resource availability.
6. Validate stability conditions.
7. Commit placement atomically.
8. Emit placement event and persistence event.
9. Replicate to relevant clients.

If any validation fails, the request is rejected and logged with reason and tick.

4. Combat event contract

```
CombatEvent
```

- eventId
- attackerId
- targetId
- weaponId
- damageAmount
- damageType
- hitLocation
- sourcePosition
- targetPosition
- tick
- serverId
- validationHash
- timestamp

Rules:

- The client can request actions, but the server generates authoritative resolution.
- Damage events are logged and tied to a server tick and world context.
- Rewind and hit validation must have an explicit bounded window.
- Invalid or impossible events are rejected and recorded as suspicious signals.

5. Save and snapshot contract

```
WorldSnapshot
- snapshotId
- worldId
- serverId
- schemaVersion
- generatedAtTick
- checksum
- previousSnapshotId
- journalStartTick
- journalEndTick
- isCommitted
```

Rules:

- Write to a temp snapshot first.
- Validate checksum before promotion.
- Only promote a snapshot if it is internally consistent.
- Never overwrite the latest valid snapshot with a partial write.
- Snapshot and event journal must be compatible with rollback.

6. Wipe contract

```
WipeOperation
- wipeId
- wipeType
- worldId
- plannedAt
- startedAt
- completedAt
- oldWorldSnapshotId
- newGeneratorVersion
- playerStatePolicy
- worldStatePolicy
- reasonCode
```

Rules:

- Wipe is a server-managed lifecycle event, not a manual file deletion.
- Any map data wipe must be tested in a dry run before production use.
- Player state migration must be deterministic and traceable.
- The wipe operation must produce an auditable log for rollback and support review.

7. Weather and time contract

```
WorldEnvironmentState
- worldId
- tick
- timeOfDay
- weatherState
- temperatureBias
- visibilityModifier
- lastTransitionTick
- deterministicSeed
```

Rules:

- Time of day and weather are authoritative server states.
- Weather changes directly affect temperature, visibility, and AI perception.
- Client visuals are derived from the authoritative state, not the other way around.

8. AI perception contract

```
AIPerceptionSnapshot
- aiEntityId
- tick
- visiblePlayers
- audibleEvents
- threats
- lineOfSightState
- lastKnownTarget
```

Rules:

- AI must use server-authoritative perception, not client-only data.
- Visibility and hearing depend on world state and environment conditions.
- Perception must be budgeted and decoupled from rendering.

32.4 Phase exit criteria (non-negotiable)

Each phase is complete only when the exit criteria are met. These are the actual completion gates, not just a list of implemented features.

Phase 0 exit criteria

- ☒ The game boots cleanly in the Unity Editor.
- ☒ Repeated local save/load cycles preserve player state and enforce schema versions.
- ☐ Interrupted-write recovery is fault-injected and preserves the last valid save.
- ☒ A deterministic RNG service passes repeatability and state-restoration tests.
- ☐ Localization keys exist for the first playable UI surface.
- ☐ Structured logs and basic telemetry are visible.

- ☒ Editor tests and a local runtime test scene are available.
- ☐ Multiplayer is not started until the offline-first gate in Section 2A is met.

Phase 1 exit criteria

- A player can gather resources, open inventory, craft a valid item, and maintain survival stats.
- Hunger, thirst, and temperature decay are server-authoritative.
- Death and respawn states are enforced from the server.
- Save integrity survives a full save/load cycle.
- Basic chunk loading and unloading work without breaking world state.

Phase 2 exit criteria

- Base placement is permission-checked and server-authoritative.
- Structures persist across save/load and remain valid after restart.
- Structure collapse or invalid support states remain within configured budgets.
- Large base scenarios remain stable under stress tests.

Phase 3 exit criteria

- Combat is resolved server-side with valid hit validation.
- Fire rate, aim state, and damage application are checked under simulated packet loss.
- Lag compensation is bounded and never uses arbitrary historical state.
- Damage events are attributable and auditable.

Phase 4 exit criteria

- The same world seed and generator version produce the same world.
- Chunk streaming, resource placement, navigation, and terrain states work deterministically.
- Validation identifies impossible terrain, unreachable spawn points, or invalid monument placement.
- Weather and day/night cycles alter gameplay state in measurable, tested ways.

Phase 5 exit criteria

- Progression and unlocking are consistent across save/load.
- Recipe validation, output generation, and unlock conditions are reproducible.
- At least one full progression loop is available from gathering to crafted outputs to survival benefit.
- Content definitions are data-driven and versioned.

Phase 6 exit criteria

- Load testing, crash recovery, and database failure recovery are exercised.
- Server health checks, metrics, and alerts are active.
- Wipe and restore paths are tested end-to-end.
- Admin and moderation flows work with logs and audit trails.

Phase 7 exit criteria

- Closed-test readiness is achieved with player privacy and data export flows validated.

- Localization and accessibility pass is complete for the launch set.
- Full wipe cycle has executed at least once in a test environment without losing data.
- production release pipeline rehearsal is complete.

Phase 8 exit criteria

- Launch monitoring metrics remain within thresholds for the first week of live operation.
- A real hotfix path is proven under pressure.
- Player report → moderator action → attributable event log is fully working.

Phase 9 exit criteria

- A recurring wipe cycle is active and repeatable.
- First live event or remote configuration-driven update is in production.
- Telemetry, balancing review, and patch planning are executed as a documented recurring loop.

Phase 10 exit criteria

- Modding API, if in scope, has an approved trust boundary and documentation.
- Community-facing tooling is tested in limited beta before public rollout.
- Platform scalability and accessibility maturity goals are formally evaluated against telemetry.

32.5 Engineering guardrails to prevent hallucination and drift

This project requires explicit anti-drift rules to keep the implementation aligned with what is real and tested.

Guardrail 1 — decisions must be written before implementation

Any major system must have a written decision record:

- player authority model
- savings and schema versioning rules
- world generation rules
- time/weather model
- combat model
- economy balancing model
- moderation authority model
- wipe lifecycle model

Guardrail 2 — no feature is complete without test evidence

A feature is not done because it appears correct in code. It is done when it passes the relevant tests and is validated against the real system contracts.

Guardrail 3 — no hidden assumptions in gameplay logic

Any implicit assumption must become a named rule. Examples:

- who owns the correct state
- what the authoritative tick is
- what range is legal for interactions
- what counts as a valid resource state
- how durability and progression are saved

Guardrail 4 — every critical state mutation must be logged

Every critical operation must have an attributed event record:

- inventory change
- combat event
- building placement
- structure destruction
- admin/moderator action
- wipe process action
- save snapshot promotion

Guardrail 5 — scope must be controlled by a known delivery gate

No phase may begin without a definition of done. If the team cannot define the exit criteria, the phase is not ready.

Guardrail 6 — the first playable build must be offline-first

The project must prove stable gameplay before adding online complexity. Multiplayer should be treated as a second-stage problem, not the first-stage proving ground.

Guardrail 7 — user-facing decisions are product decisions, but technical execution is engineering truth

If a balance decision is made by the design team, it still must be converted into a technical, testable contract with exact rules. Product intent is not enough; the system must encode it into validation logic.

33. Production Readiness Checklist

Before public launch:

Architecture

- ☐ Server authority documented
- ☐ Tick model documented
- ☐ Ownership boundaries documented
- ☐ Failure modes documented
- ☐ Engine decision finalized (Section 2)

Networking

- ☐ Prediction/reconciliation tested
- ☐ Interest management profiled
- ☐ Packet validation implemented
- ☐ Rate limits implemented

Survival & Environment

- ☐ Vitals decay verified under server authority
- ☐ Weather affects temperature and visibility, not just VFX
- ☐ Day/night cycle single-source-of-truth verified

Building

- ☐ Stability tested against reference implementation
- ☐ Large structure limits defined
- ☐ Collapse workload budgeted
- ☐ Permissions authoritative

World

- ☐ Seed determinism verified
- ☐ Generator version stored
- ☐ Chunk lifecycle implemented
- ☐ World validation implemented

Inventory/Crafting

- ☐ Atomic transactions
- ☐ Item definitions versioned
- ☐ Duplicate-item protection
- ☐ Recipe migrations tested

Combat

- ☐ Server-side hit resolution
- ☐ Timestamp validation
- ☐ Bounded rewind
- ☐ Fire-rate validation
- ☐ Penetration tests

Vehicles

- ☐ Driver prediction/reconciliation tested
- ☐ Passenger sync verified under packet loss
- ☐ Vehicle physics budget enforced

Audio

- ☐ Gameplay sound events server-authoritative

- ☐ Cosmetic audio never touches network
- ☐ Occlusion behavior decided and implemented if in scope

Persistence

- ☐ Versioned schemas
- ☐ Atomic snapshots
- ☐ Checksums
- ☐ Backup strategy
- ☐ Crash recovery
- ☐ Restore drills completed
- ☐ Wipe orchestration rehearsed end-to-end

Security

- ☐ Secrets excluded from clients
- ☐ Authentication implemented
- ☐ Authorization implemented
- ☐ Abuse/rate limiting
- ☐ Anti-cheat telemetry

Legal & Compliance

- ☐ Data export/deletion request path tested end-to-end
- ☐ Age-rating content gating implemented
- ☐ EULA/ToS enforcement tied to moderation tooling

Onboarding & Accessibility

- ☐ Tutorial completion state server-tracked and versioned
- ☐ Launch-language localization complete
- ☐ Colorblind-safe indicators and remappable controls shipped

Operations

- ☐ Health checks
- ☐ Metrics
- ☐ Alerts
- ☐ Crash reporting
- ☐ Deployment rollback
- ☐ Server draining
- ☐ Staged rollout pipeline rehearsed
- ☐ Hotfix path rehearsed

Admin & Moderation

- ☐ Admin panel functional for real moderator workflows
- ☐ Player report → review → action loop tested end-to-end

- ☐ Moderator actions attributable per Rule 4

Testing

- ☐ Unit tests
- ☐ Integration tests
- ☐ Property tests
- ☐ Determinism tests
- ☐ Load tests
- ☐ Soak tests
- ☐ Failure injection

34. Key Risks and Mitigations

Risk	Impact	Mitigation
Client authority	Critical	Server-owned state and validation
Nondeterministic world generation	Critical	Seed/version contracts + automated determinism tests
Save corruption	Critical	Atomic snapshots + checksums + backups
AI-generated concurrency bug	Critical	Human review + race/failure testing
Lag compensation exploit	Critical	Bounded rewind + server timestamp validation
Vehicle passenger desync	High	Server-parented passenger state, not independent prediction
Large base performance	High	Incremental stability + workload budgets
Network replication overload	High	Interest management + prioritization
Item duplication	Critical	Atomic inventory transactions + unique IDs
Schema incompatibility	High	Versioned migrations
Wipe orchestration failure	Critical	Rehearsed end-to-end wipe drill before first live wipe
Modding trust-boundary breach	Critical	Sandboxed hook APIs, no raw persistence access
AI over-abstraction	Medium	Simplicity review and explicit contracts
Content production bottleneck	High	Data-driven tools and editor pipelines
Operational blind spots	High	Metrics, logs, alerts and dashboards
Server crash recovery failure	Critical	Automated restore/soak testing

Risk	Impact	Mitigation
Economy imbalance	Medium/High	Recipe unrolling and telemetry
Procedural world quality issues	Medium	Automated validation + human playtest
Late localization/accessibility retrofit	Medium/High	Infrastructure built in Phase 0, content completed by Phase 7
Data-privacy request unsupportable	Critical	Player-data domain separation from Phase 0
Post-launch content drought	High	Live-ops cadence established explicitly as Phase 9, not assumed

35. AI Usage Matrix

System	AI usefulness	Human ownership required
Architecture	Medium	Very high
Network protocol	Medium	Very high
Movement prediction	High	High
Stability	High	Very high
World generation boilerplate	Very high	High
World tuning	High	Very high
Survival vitals/status effects	High	Medium
Weather/day-night	High	Medium
Item definitions	Very high	Medium
Inventory transactions	High	Very high
Crafting	Very high	High
Combat implementation	High	Very high
Lag compensation	Medium	Extremely high
Vehicle physics	Medium	High
Vehicle passenger sync	Medium	Very high
Sound event relevance system	High	Medium
Persistence serialization	Very high	Very high
Snapshot concurrency	Medium	Extremely high

System	AI usefulness	Human ownership required
Wipe orchestration scripting	High	Very high
Pooling	Very high	Medium
Interest management	High	High
AI/NPC behavior	High	High
Modding hook scaffolding	High	Very high
Modding sandbox/security	Low	Extremely high
Admin panel tooling	Very high	Medium
Localization scaffolding	Very high	Low
Tests	Extremely high	High
Documentation	Extremely high	Medium
Editor tools	Extremely high	Medium
Profiling tooling	High	High

36. Recommended Engineering Backlog

The project should not begin by implementing all systems independently — build the dependency chain in order:

```

Offline Foundation
↓
Core Survival Loop
↓
Building
↓
Combat and Procedural World
↓
Progression and Content
↓
Scale, Hardening, and Production Readiness
↓
Launch Stabilization
↓
Authoritative Networking
↓
Live-Ops Cadence
↓
Platform Maturity (Modding Rollout, Accessibility Completeness, Scale)

```

This order prevents teams from building gameplay systems on unstable contracts, and extends the original six-system dependency chain through launch into an ongoing operating model.

37. Final Engineering Principles

1. **AI accelerates implementation; humans own architecture and correctness.**
2. **The server owns all meaningful game state**, including vitals, sound events, and vehicle physics — not just the original six systems.
3. **Determinism is an architectural contract, not a coding preference.**
4. **Every persistent schema must be versioned.**
5. **Every critical transaction must be atomic.**
6. **Every expensive system needs an explicit workload budget.**
7. **Every important gameplay action — including moderator and admin actions — should be observable and attributable.**
8. **Every correctness-critical system should have a slow reference implementation or strong invariant tests where practical.**
9. **Do not optimize from AI guesses; profile real workloads.**
10. **Do not introduce abstractions without a demonstrated requirement** — including in planning documents themselves; scope every phase to the team size actually executing it.
11. **Do not retrofit networking, persistence, security, observability, localization, or modding hooks after gameplay is complete.**
12. **Test failure conditions — including wipes and moderation workflows — as aggressively as successful gameplay.**
13. **Treat world generation, persistence, and server simulation as versioned products, not disposable code.**
14. **Use AI heavily for repetitive, well-specified work and conservatively for security-, concurrency-, and authority-critical work.**
15. **A system is production-ready only when its correctness, failure behavior, performance envelope, persistence behavior, and operational visibility are understood.**
16. **Launch is a milestone, not an end state** — the roadmap continues through live-ops cadence and platform maturity, and should be resourced as such from the start rather than assumed to wind down at ship.

The central conclusion remains: the highest-value AI strategy is not to have AI "write the game," but to use AI as a force multiplier for implementation, reference implementations, tests, tooling, and iteration, while senior engineers retain ownership of the decisions that determine correctness and long-term scalability — through launch and for as long as the game operates afterward.